

ECDAT — Solution Pipeline

Enterprise Cryptographic Discovery & Analysis Tool · SIH26164 · NTRO · Internal design note

This document describes the full pipeline as decided: how a scan is started, how the user controls what gets read, what each analysis stage does, and how mismatches between live scan results and configuration files are reported.

1. Scan modes

Two entry points. Both run the same pipeline and write to the same store, so results are comparable regardless of how the scan was started.

Mode 1 — Network probe only

The user supplies a host or URL. Nothing is uploaded, no files are read. The probe performs TLS handshakes against the live endpoint and everything downstream runs on those findings alone.

The alignment check produces no output in this mode, because there are no configuration files to compare live results against. The UI should state this explicitly rather than leaving the section silently blank — otherwise users will assume the feature is broken.

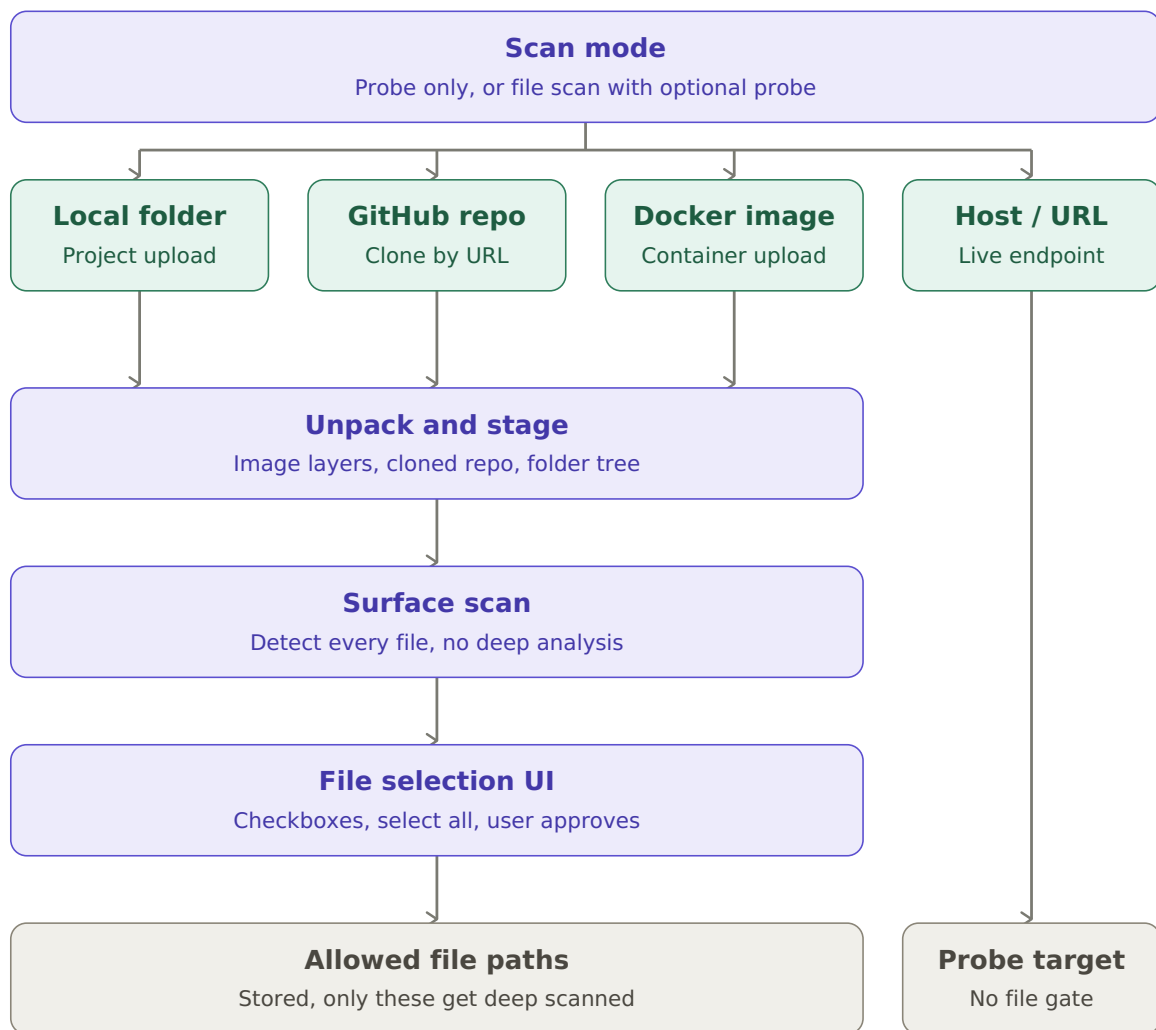
Mode 2 — File scan, with optional network probe alongside

The user uploads a local folder, a GitHub repository, or a Docker container image. They may additionally supply a host or URL, which is probed in the same run.

This is the mode where the alignment check becomes meaningful: the file scan supplies what the configuration *declares*, the probe supplies what the server *actually does*, and the two can be compared for the same asset in the same run.

The probe host is entered explicitly by the user alongside the upload. It is not inferred from scanned files.

2. Intake and permission



Green = user input, purple = intake steps, grey = handed to the scan pipeline

Figure 1 — Intake and permission flow

Unpack and stage. Three of the four input types become a directory tree. A local folder already is one; a repository gets cloned; a Docker image gets unpacked layer by layer. After this step all three look identical to everything downstream — one filesystem on disk.

Surface scan. Walk the tree and list every file. Fast, no parsing, no analysis. Its only job is producing the inventory the user is about to approve.

File selection UI. The list rendered as checkboxes with select-all and the usual bulk controls. The user unchecks anything they do not want touched — files holding secrets, anything out of scope.

Store allowed paths. The approved addresses are persisted. Every collector in the analysis pipeline reads from this list and nothing else. A file the user unchecked is never opened.

The Host/URL input bypasses all of this. There are no files to select — the target is a live endpoint, and the only thing read is what the server sends over the wire during a handshake. This is why the probe is available as a standalone mode.

3. Analysis pipeline

Everything below runs on the approved file list and, when supplied, the probe target.

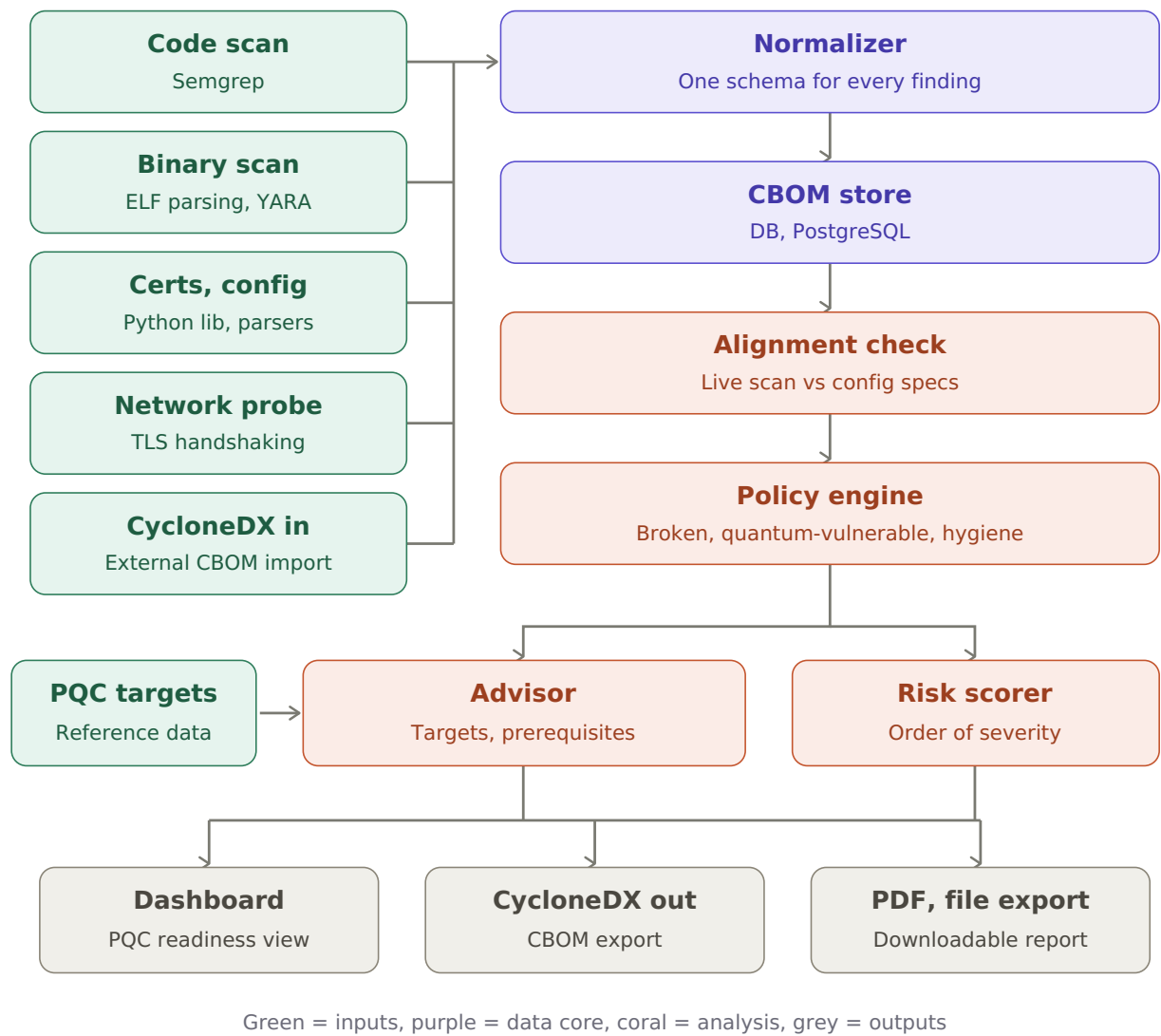


Figure 2 — Analysis pipeline

Collectors

Four read from the approved file list — Simgrep over source, ELF parsing plus YARA over compiled binaries, the Python cryptography library over certificates, and format-specific parsers over configuration files. The fifth, the network probe, reads nothing from disk; it handshakes against the Host/URL target. A sixth path, CycloneDX in, accepts a CBOM produced by an external tool.

Normalizer

Five collectors return five incompatible output shapes. This maps all of them onto one finding schema, so nothing downstream needs to know which collector produced what.

CBOM store

PostgreSQL, own schema. CycloneDX is the format imported and exported at the edges, not the format stored in.

Policy engine

Classifies every finding as broken, quantum-vulnerable, or a hygiene issue.

Advisor

Takes classified findings and produces replacement targets plus the prerequisites needed to deploy them, backed by the

PQC targets reference data.

Risk scorer

Orders findings by severity.

Outputs

All three read from the same analysis results — the dashboard for the PQC readiness view, CycloneDX for machine-readable export, and PDF or other file formats for a downloadable report.

4. Alignment check

This stage compares what the live scan observed against what the configuration files declare. It only produces output when both are present, which means Mode 2 with the optional probe enabled.

Two edge cases arise, and to the system they look identical — the algorithm observed live differs from what the config declares.

Case 1. The configuration declares one thing and the implementation uses another. A genuine mismatch.

Case 2. The configuration declares one algorithm for a use, but among the places that use it, some legitimately require something else for their particular case. For example, five usage sites covered by a single config declaration, of which two need a different algorithm.

The rule, for both cases

The result is reported with respect to what the live scan observed. A note is attached stating that this does not align with the configuration specification for that asset.

Flagging is per usage site, not per configuration declaration. In the Case 2 example, only the two sites that differ are flagged. The three that match the declaration are not flagged at all. Whether a flagged mismatch is legitimate is left to the user to decide — the tool reports the divergence and does not attempt to classify it.

This keeps the logic simple. The system never has to work out *why* the live result differs, only *that* it differs, and exactly where.
